LoDA-SA: Current configuration and evidence

Supplementary Note S1: Current shared-coordinate configuration

The selected operating point uses $(n_b, n_l) = (32, 4096)$ with token quotas off, not a maximum selected independently from each benchmark. For Qwen3-8B split-half RoPE, $J = \{48, \dots, 63, 112, \dots, 127\}$ contains 16 complete low-frequency pairs. One width-32 view serves both selection stages; original Q/K/V remain full-dimensional for final attention. Near and far scoring share the map. The extrema upper bound applies only in this view, not to original-dimensional logits or page attention mass. Pairing semantics do not automatically extend to learned projections, static-pair modes or MLA.

The reference path disables dynamic far gating, static layer skipping and L2 candidate gathering. L2 computes full-history logits before page masking, per-query-head softmax and KV-group probability averaging, using scale $1/\sqrt{128}$. Final selected-KV access is backend-dependent. Configuration labels in quality snapshots do not certify all runtime flags or a clean producer tree.

Supplementary Figure S2: Regional page allocation and single token pool



Under sufficient eligible capacity, $K_1 = 128$ gives ranked-page quotas 32 near and 96 far. The 1024-token cap includes four sink and 128 recent tokens, leaving 892 nominal pooled slots when protection is disjoint. Boundary bypass pages enter outside ranked quotas; up to four protected prefix positions can be reselected, reducing distinct unprotected coverage. The blocks are schematic; per-query normalization precedes KV-group probability averaging.

Supplementary Note S3: Integer boundaries and support semantics

Integer boundaries and summary support. In the current explicit-n_l/n_b partitioned path, with $S = t+1$ valid cached positions including the current token and excluding padding, let $M = \max(0, S - s - w)$, $L = \max(B, n_l)$ and

$$b_n = \max(\lfloor s/B \rfloor, \lfloor (S - w - L)/B \rfloor),$$
$$b_w = \max(b_n, \lceil S/B \rceil - \max(1, \lfloor w/B \rfloor)).$$

For sufficient history and disjoint protection, far and near L1 intervals before removing bypass pages are $[\lfloor s/B \rfloor, b_n)$ and $[b_n, b_w)$, respectively. The unified L2 interval is $[B\lfloor s/B \rfloor, S - w)$, with the combined page mask. Rounding and alignment can change near length from n_l. Summaries use full physical pages, not unprotected intersections: the partial prefix page includes protected

keys, and the last near page can include recent-window keys at nonaligned S. Pooled L2 still stops at $S - w$. These rules exclude clustering, quota overrides and short-history branches.

Scarce retained support. In current unified selection, L1 page masking precedes L2 softmax. Pooled TopK clips counts to interval width, not positive support. If the clipped count exceeds that support, zero ties can select cached tokens from unselected pages; final selection does not reapply L1 eligibility. Figure S2 assumes sufficient support and disjoint protection; partial-prefix reselection is separate. Occurrence and quality effects in the reported runs remain unverified.

Fair comparison. A matched comparison would require identical model, input IDs, generation limits and task scorer; equal-K sparse comparisons count protection within the final unique cap. Retain and report each method's native phase, layer and protection policies; policy-matched adapters are separate controlled comparisons. Internal page caps may differ, but construction, scoring, ranking and traffic count toward complete-request cost. Report realized token counts and selection granularity, and match hardware and precision for timing.

Supplementary Note S4: Current-configuration evidence

Pooled-quality records (T6). At revision `a7bcdddf727aa99196d925b9f106d785ce9273d4`, the manuscript input archive contains three T6 result JSONs, one for each length (32768, 65536 and 131072); their filenames use the prefix `t6_ruler_q8b_L` and suffix `_champ.json`. Their 11-task arithmetic means are 59.0209, 47.8000 and 46.8164; differences from the existing dense anchors are -0.3609, -0.7364 and $+0.3845$ points. Every task records $n = 100$, with no incomplete cells. The source-defined labels bind $B = 64$, $K_1 = 128$, $K = 1024$, compression ratio 4, 4/128 protection and $(n_b, n_l) = (32, 4096)$ with token quotas off. Provenance notes batch sizes 8/2/2 and YaRN off/2/4 across lengths. The snapshots store prediction-file hashes, not matching scorer, source-input or full effective-configuration manifests; those hashes cannot establish paired data or execution identity. The exact reported configuration string is `tli_64_128_1024_c4_Aa0_b0_g1_sk4_sw128_np32_ntoff_nl4096`. The three result JSONs are available in the versioned manuscript directory `figures/ruler32k_comparison/inputs/`. Each result binds task scores, counts and prediction-file SHA-256 values. Separate receipts binding producer script, seed and generation identity have not been recovered in this inspected tree; those fields are not certified here. The following three tables map all 33 plotted differences to task names, dense scores and pooled scores. No paired uncertainty or dense execution matching follows from these result records.

32K task scores

Each reported task has 100 examples per arm. Differences are pooled minus dense, computed before rounding.

Task	Dense	Pooled	Difference
niah_single_1	100.00	100.00	+0.00
niah_single_2	100.00	100.00	+0.00
niah_single_3	100.00	100.00	+0.00
niah_multikey_1	99.00	98.00	-1.00
niah_multikey_2	76.00	71.00	-5.00
niah_multikey_3	4.00	7.00	+3.00
niah_multiquery	24.50	24.50	+0.00
niah_multivalue	25.00	28.50	+3.50
cwe	83.50	74.10	-9.40
fwe	7.00	15.33	+8.33
vt	34.20	30.80	-3.40

64K task scores

Each reported task has 100 examples per arm. Differences are pooled minus dense, computed before rounding.

Task	Dense	Pooled	Difference
niah_single_1	100.00	94.00	-6.00
niah_single_2	100.00	100.00	+0.00
niah_single_3	100.00	100.00	+0.00
niah_multikey_1	100.00	99.00	-1.00
niah_multikey_2	50.00	39.00	-11.00
niah_multikey_3	0.00	0.00	+0.00
niah_multiquery	20.25	22.50	+2.25
niah_multivalue	26.25	36.00	+9.75
cwe	0.60	0.30	-0.30
fwe	4.00	5.00	+1.00
vt	32.80	30.00	-2.80

128K task scores

Each reported task has 100 examples per arm. Differences are pooled minus dense, computed before rounding.

Task	Dense	Pooled	Difference
niah_single_1	100.00	97.00	-3.00
niah_single_2	100.00	100.00	+0.00
niah_single_3	100.00	99.00	-1.00
niah_multikey_1	99.00	99.00	+0.00
niah_multikey_2	57.00	39.00	-18.00
niah_multikey_3	1.00	0.00	-1.00
niah_multiquery	12.25	17.00	+4.75
niah_multivalue	2.50	3.75	+1.25
cwe	3.40	0.70	-2.70
fwe	13.00	37.33	+24.33
vt	22.60	22.20	-0.40

Graph execution records (E152v4). The same revision archives `exp/trace/results/e152v4_formal_pc40/` and its verdict. Recomputed dense/TLI Graph mean TPOTs are 26.282/24.702, 39.670/27.398 and 67.035/32.425 ms at nominal 32K/64K/128K. For example, the 128K timing record contains 134923 input tokens after prompt construction. The protocol reports Qwen3-8B BF16 on H20-3e, batch one, ABBA process ordering, two processes per arm and fixed 128-token output; dense has six measured repeats and Graph four per length. Per-step synchronization and argmax are included. Graph TPOT divides the post-cache-fill decode wall by 127, excluding static-cache filling and warmup capture. It does not report a preparation-inclusive request wall. The six dense files instead use the E152 v2 schema, DynamicCache and post-first-output timing; their records do not establish Graph execution. Although the later Graph script supports both arms, that capability cannot relabel these saved dense runs. Both arms mark backend verification false, and the declared runtime checkout does not supply the archived producer script, so exact runtime producer identity remains unresolved. The published v2 script calls model forward in a loop, not `generate`; this is inspected code, not complete runtime verification. Graph/eager token agreement is reported on the timing prompt, not benchmark-wide equivalence or equality to FullKV. The Graph treatment agrees with the T6 labels on the explicit budgets above, but T6 lacks a full frozen manifest and uses different batches. Neither the quality/timing records nor their speed ratios define a matched quality–cost frontier.

Environment scope. The verdict designates the primary H20-3e environment before its length-wise summary. All its measured runs are used. A separate local-GPU1 replica reports dense/sparse Graph TPOTs of 39.338/47.605, 59.250/51.572 and 77.831/66.513 ms at 32K/64K/128K (six/four measured runs per length). The 32K dense/sparse ratio is .826: the selected method is slower in that record. The source labels this replica co-tenant-polluted; independent interference logs have not been verified, so that attribution is reported rather than established. These results are not pooled across environments and do not support a hardware-independent acceleration claim.

Cross-model execution diagnostics (E159/E160). Table II in the main text is recomputed from raw records at revision `906d81219a2f55ecf7a73d1851ccb96655c0b585`. Under `exp/trace/results/`, E159 uses `e159_llama/e159_phase1_v2_pc40.json` for dense eager and `e159_phase2_graph_pc40.json` for sparse Graph. E160 uses the `dense_L{32768,65536}_b{1,2}.json` and corresponding `tli_graph_L{32768,65536}_a{1,2}.json` records in `e160_32b/`. Means exclude marked warmups and weight measured runs equally. Each row has matching recorded 16-hex-digit input-hash prefixes across arms; full hashes and a complete input/model/tokenizer manifest are not supplied. E159 draws its two arms from

separate phase archives, not one verified cross-mode interleaving. Llama has six measured runs per arm/length; Qwen3-32B has six dense and four sparse runs. These repeats share a timing prompt, not independent task examples. A second-device Qwen3-32B record (`e160r_32b/`) gives dense/sparse means 55.582/54.685 and 80.414/59.969 ms at 32K/64K, respectively; it is a replication of timing, not quality.

The configured route is BF16 Hugging Face/PyTorch SDPA on H20-3e with Torch 2.8.0 and Transformers 4.56.2, not a verified production-serving backend. Sparse labels bind $K_1 = 128$, $K = 1024$, compression ratio 4, 4/128 protection, near pages/extent $(32, 4096)$ and token quotas off. Source provenance reports a shadow implementation, so a checkout identifier alone does not fix the executed code. Sparse TPOT measures the 127 post-fill decode intervals, including synchronization and argmax but excluding prefill, cache filling and Graph capture. Dense uses eager post-first-output timing. Thus the table compares execution regimes, not equally optimized Graph backends. The E159 dense-Graph path is markedly slower than dense eager; its 14.28 ratio at 128K is not used as evidence of the selector's advantage. Llama Graph/eager output agreement is checked only on a separate 16362-token smoke prompt. Qwen3-32B saved Graph/eager token arrays agree on the timing prompts; neither output-agreement check establishes task quality against FullKV.

Qwen3-32B 128K protocol change (E160b). The table's final row uses `e160b_128k/remote_e160b/`: two `dense_L131072_b{1,2}.json` records and four `tli_graph_L131072_a{1,2,3,4}.json` records. Both arms enable the expandable-segment allocator. Sparse runs use four separate processes, each with one measured run and no discarded warmup; capture occurs in that run but lies outside its reported TPOT. Dense retains one warmup and three measured runs per process. Recomputed means are 129.100739/70.109988 ms, with ratio 1.841403; statistical equivalence to the earlier repetition protocol is not established. All four sparse outputs match the single eager output-agreement run. Actual prompt length is 134923, exceeding the nominal 131072 label; the logs also warn about this length, and the result does not validate long-context quality. In the original E160 logs, warmup decoding succeeds before the next prefill fails with an out-of-memory error. The source report states that a repeated-run attempt with expandable segments also fails after successful warmup; the single-run-process records establish a narrower feasible execution, not a general memory-capacity guarantee. No preparation-inclusive request savings or matched quality–cost frontier follows from these timings.